

DESAFÍO TÉCNICO DE PRÁCTICA

Construcción de soluciones sobre Sparx Enterprise Architect y Prolaborate

Pasantías Proagile 2026



Informe Ejercicio 1 - Creacion de Addin



Ezequiel Pino

Informe Principal – Addino: Revisión de Metadatos para Enterprise Architect

Introducción	3
Inicio de Desarrollo	4
Primer Add-in de prueba	4
Desarrollo asistido por Ecosistema Gentle-AI	5
Tecnologías utilizadas	6
Diseño de la solución	6
Integración con Enterprise Architect	7
Carga de elementos	8
Modelo local de edición	8
Persistencia controlada	9
Manejo de errores	10
Archivos principales de la solución	10
Validación funcional	11
Verificación técnica	11
Evidencia y documentación complementaria	12
Conclusión	13

Introducción

En el contexto planteado, los elementos modelados en Enterprise Architect contienen **información como nombre, alias, notas, tipo y estereotipo**. Parte de esta información es utilizada posteriormente en reportes, búsquedas, análisis de trazabilidad y comunicación entre equipos, por lo que resulta importante mantenerla completa y consistente.

El problema principal es que, sin una herramienta adicional, **esta revisión requiere abrir y modificar cada elemento individualmente dentro de Enterprise Architect**. La solución desarrollada busca centralizar esta tarea mediante una única interfaz que permita consultar y modificar los principales metadatos de los elementos pertenecientes a un paquete.

Como resultado se desarrolló **Addino**, un Add-in en C# para Sparx Enterprise Architect que centraliza la revisión y actualización de metadatos de los elementos contenidos en un paquete y en toda su jerarquía de subpaquetes.

	Nombre	Alias	Notas	Tipo	Estereotipo	Paquete
	S		dsadas	Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 10		sdaasd	Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 11		adssad	Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 12		adasd	Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 13		ads	Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 14			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 15			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 16			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 17			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 18			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 19			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 2			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 20			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 21			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 22			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 23			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 24			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 25			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 26			Class	ArchiMate_Applic...	Aplicaciones
	Aplicación 27			Class	ArchiMate_Applic...	Aplicaciones

Permite visualizar y editar Nombre, Alias y Notas; consultar Tipo, Estereotipo y la ruta Paquete; identificar modificaciones pendientes; validar Nombres antes de guardar; cargar información desde Enterprise Architect y descartar cambios de forma segura. A continuación, adjunto un video Guía para la ejecución de Addino:

Link Video: Pino Guia Ejecucion Addino

El objetivo principal de Addino es **facilitar la revisión y actualización de los metadatos de los elementos contenidos dentro de un paquete de Enterprise Architect**. En la fase inicial, que cumple con el alcance mínimo exigido por el desafío, incluye:

Desde una única grilla es posible visualizar los elementos directos de un paquete y modificar:

- **Nombre**
- **Alias**
- **Notas**

Mientras que los siguientes campos se muestran únicamente como información:

- **Tipo**
- **Estereotipo**

Los cambios se mantienen localmente mientras se trabaja sobre la ventana y solamente se escriben en el repositorio cuando se presiona **Guardar**.

El desarrollo comenzó cubriendo primero los requisitos obligatorios y posteriormente incorporó los requisitos opcionales seleccionados.

Inicio de Desarrollo

Antes de implementar la solución definitiva, **decidí familiarizarme con el mecanismo de Add-ins de Enterprise Architect**. De esta forma tuve mi primer acercamiento al entorno donde iba a desarrollar este primer ejercicio.

Primer Add-in de prueba

Como primer paso seguí la guía de creación de Add-ins proporcionada para el desafío. Creé un proyecto básico en Visual Studio utilizando:

- C#
- .NET Framework
- Interop.EA
- Registro COM
- Enterprise Architect

La primera versión implementaba únicamente un menú de prueba, como el de la guía, con acciones simples como:

- Say Hello
- Say Goodbye

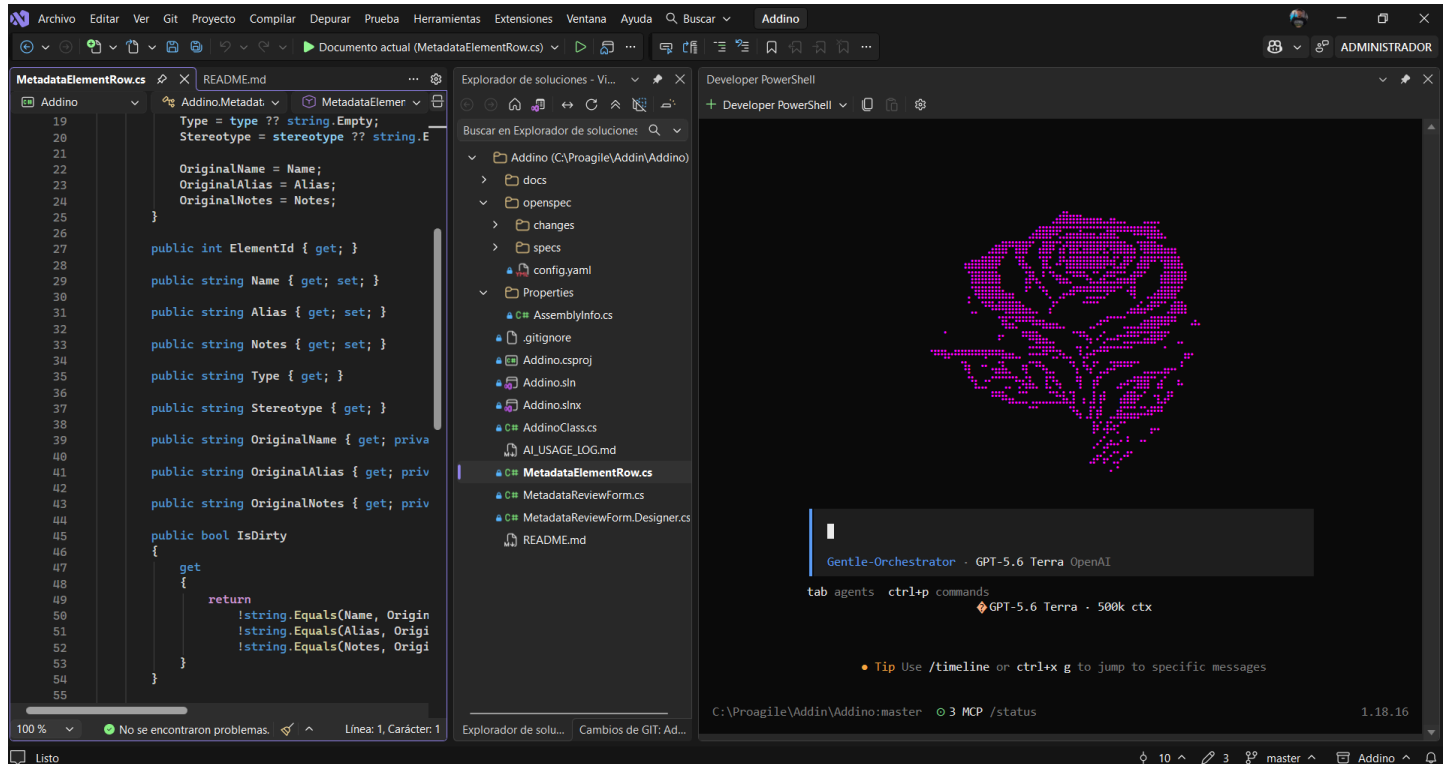
Esta implementación me permitió comprobar previamente que:

- Enterprise Architect podía cargar correctamente mi ensamblado.
- La clase COM estaba registrada.
- Los callbacks requeridos por EA fueron ejecutados.
- El Add-in aparecía correctamente en el menú de extensiones.
- Visual Studio podía compilar y registrar el proyecto en mi entorno.

Una vez validado este entorno base, utilicé ese mismo proyecto como punto de partida para desarrollar Addino.

Desarrollo asistido por Ecosistema Gentle-AI

Para organizar el desarrollo utilicé **Gentle AI**, integrado en mi entorno de trabajo con OpenCode, en su forma de TUI. Ejecutado directamente en la terminal de Visual Studio.



A continuación desarrollo brevemente como funciona un poco este ecosistema SDD con Gentle-AI, pero el detalle completo de las interacciones con herramientas y modelos de IA, así como el detalle de las mismas fases SDD, se encuentra documentado por separado en el archivo:

[Pino_Registro_Uso_IA_Ejercicio_1.pdf](#)

Al comenzar, en lugar de solicitar directamente la generación completa de la solución, utilicé un enfoque de desarrollo guiado por especificaciones (**SDD — Spec Driven Development**).

El trabajo se dividió en distintas etapas:

1. Exploración del proyecto y del desafío.
2. Definición de alcance.
3. Especificación de requisitos.
4. Diseño técnico.
5. División del trabajo en tareas.
6. Implementación por unidades pequeñas.
7. Verificación y corrección de problemas encontrados.
8. Validación mediante Human Gates en Enterprise Architect (testeo manual en EA).
9. Verify final.
10. Archive y consolidación del Spec.

El objetivo de este enfoque fue mantener trazabilidad entre:

Consigna → **Exploración** → **Spec** → **Design** → **Tasks** → **Implementación** → **Human Gates** → **Verify** → **Archive**

Las funcionalidades opcionales fueron desarrolladas incrementalmente y cada incremento, junto con su respectivo artefacto, fue validado manualmente en Enterprise Architect antes de continuar con el siguiente. Durante la implementación también se realizaron correcciones a partir de pruebas reales. Por ejemplo:

- Configuración correcta de edición multilínea para **Notas**;
- Comportamiento de la tecla **Enter** dentro de Notas;
- Cierre seguro del formulario mediante **Escape**.

Tecnologías utilizadas

La versión final de Addino fue desarrollada utilizando:

Tecnología	Uso
C#	Lenguaje principal
.NET Framework 4.7.2	Framework de ejecución
Windows Forms	Interfaz gráfica
Interop.EA	Integración con Enterprise Architect
COM	Registro y carga del Add-in
Visual Studio	Desarrollo y compilación
Enterprise Architect 17.1 x64	Entorno utilizado para las pruebas
Git	Control de versiones
Gentle AI / OpenCode	Orquestación y asistencia durante el desarrollo
Windows DWM API	Personalización best-effort de la caption nativa

La solución fue validada específicamente utilizando **Enterprise Architect 17.1 x64**.

La edición Trial de Enterprise Architect no constituye un requisito propio de Addino.

Diseño de la solución

Se decidió mantener una arquitectura pequeña y proporcional al alcance del ejercicio. La solución principal se divide en tres responsabilidades:

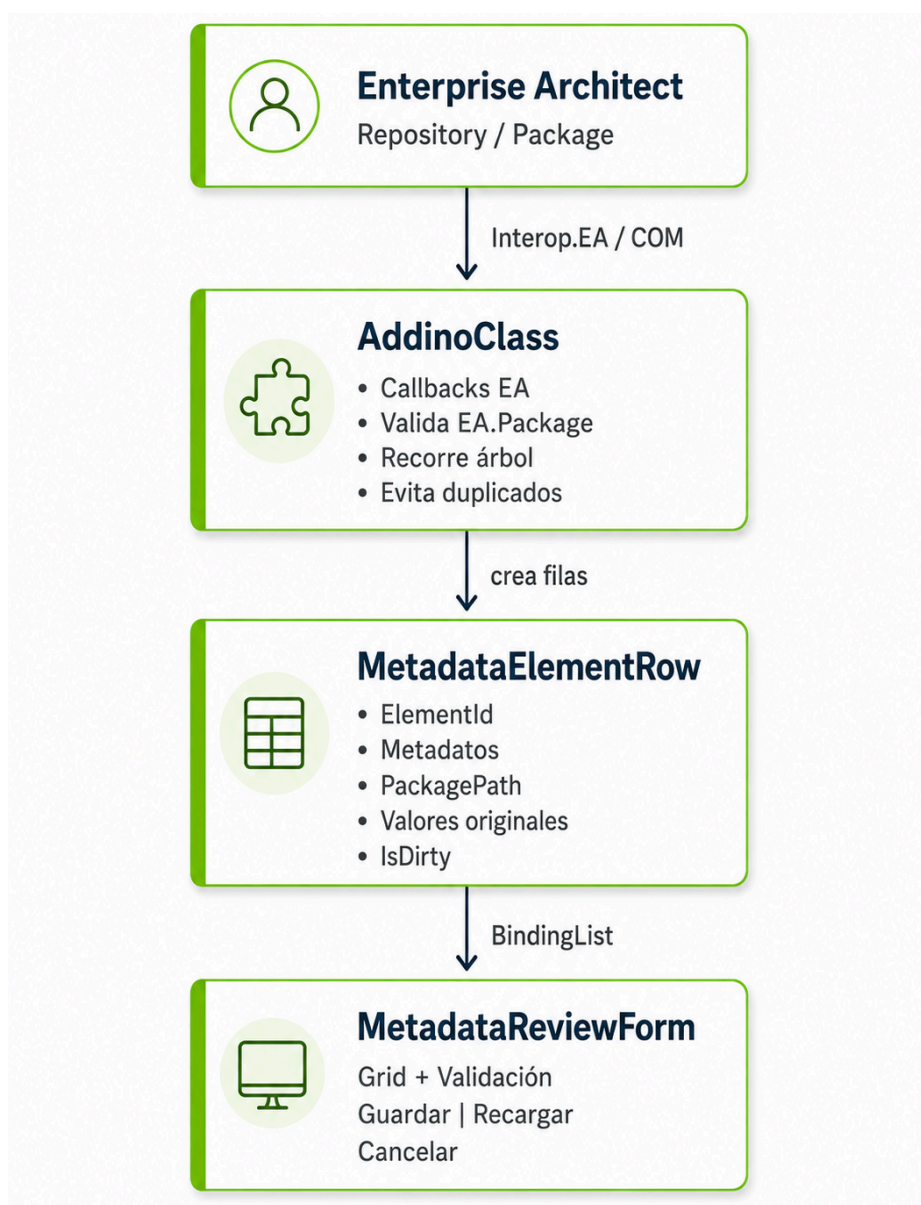
AddinoClass: punto de entrada COM, callbacks de Enterprise Architect, validación de selección y carga recursiva del árbol de paquetes.

MetadataElementRow: DTO local que almacena los metadatos, valores originales, estado IsDirty y ruta derivada PackagePath.

MetadataReviewForm: interfaz WinForms responsable de edición, validación de Nombre, indicadores visuales, Guardar, Recargar, Cancelar y presentación de errores.

Esta separación evita vincular directamente la grilla a objetos COM de Enterprise Architect. El diseño establece expresamente que el DTO almacene valores simples y el identificador estable ElementId, pero no conserve objetos EA.Element dentro de la interfaz.

El flujo general puede representarse como:



El diseño utiliza una única clase COM de entrada, un modelo local sencillo y un formulario WinForms, evitando incorporar capas o abstracciones que no fueran necesarias para el tamaño del ejercicio.

Integración con Enterprise Architect

Addino conserva los cinco callbacks requeridos para la integración:

- EA_Connect
- EA_GetMenuItems
- EA_GetMenuState
- EA_MenuClick
- EA_Disconnect

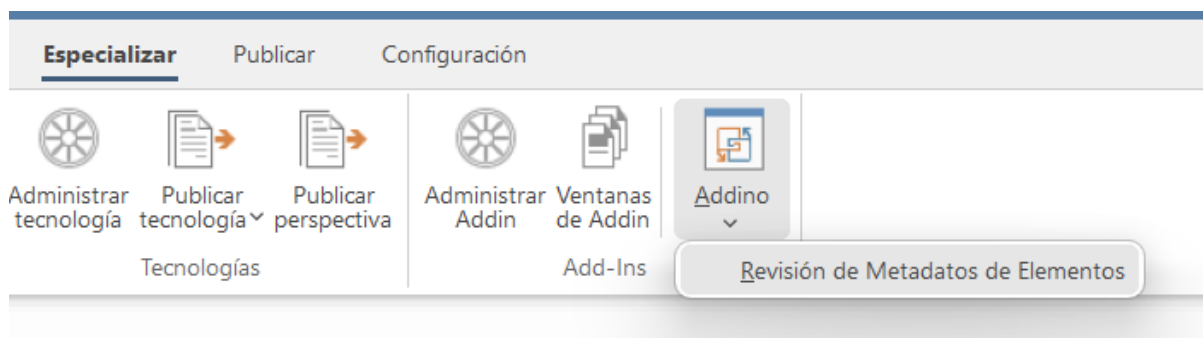
La antigua funcionalidad de prueba fue reemplazada por una única acción destinada a la revisión de metadatos. El usuario accede actualmente mediante:

Especializar → Addino → Revisión de Metadatos de Elementos

Al ejecutar esta acción, Addino inspecciona el objeto seleccionado en el Project Browser.

Para evitar aceptar incorrectamente un elemento cuyo contenedor sea un paquete, la implementación primero comprueba que el tipo seleccionado sea realmente EA.ObjectType.otPackage y luego recupera y valida el objeto seleccionado como EA.Package.

Si esta validación falla, se muestra un mensaje claro en español y el formulario no se abre.



Una vez validado, el paquete seleccionado se utiliza como raíz del recorrido de elementos y subpaquetes.

Carga de elementos

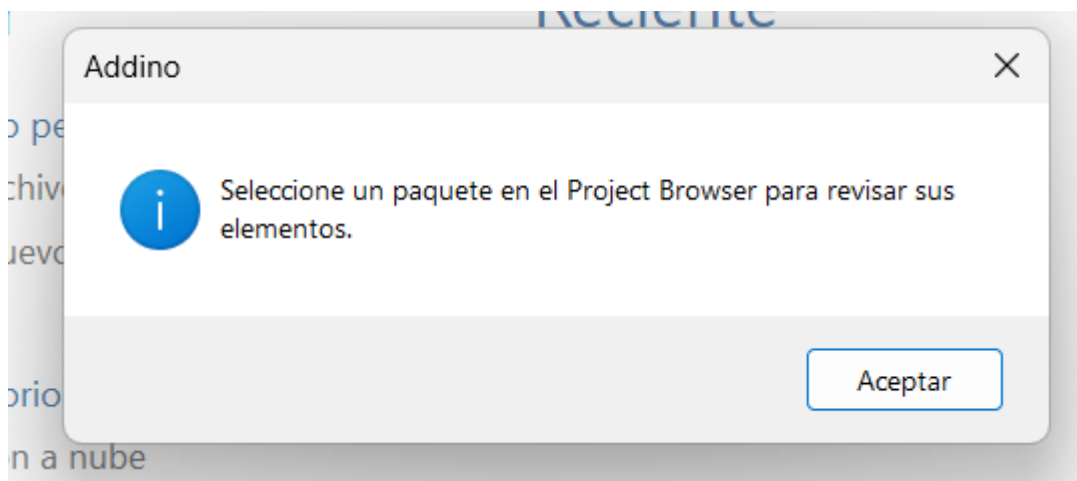
Una vez validado el paquete, Addino obtiene los elementos mediante: **EA.Package.Elements**

Una vez validado el paquete seleccionado, Addino lo utiliza como raíz de un recorrido recursivo sobre toda su jerarquía.

La carga incluye:

- elementos contenidos directamente en el paquete raíz;
- elementos contenidos en todos sus subpaquetes descendientes;
- recorrido sin límite funcional arbitrario de profundidad;
- protección frente a paquetes repetidos o ciclos mediante identificadores de paquete;
- protección frente a elementos duplicados mediante ElementID.

Para cada elemento se deriva además una ruta PackagePath, mostrada en la interfaz mediante la columna **Paquete**. Si el paquete raíz y todos sus descendientes carecen de elementos, la ventana puede abrir normalmente mostrando una grilla vacía.



Si el paquete no contiene elementos, Addino puede abrir igualmente la ventana mostrando una grilla vacía sin producir un error.

Modelo local de edición

Cada elemento cargado se transforma en un objeto MetadataElementRow. Este objeto almacena:

- ElementId;
- Name;
- Alias;
- Notes;
- Type;
- Stereotype.
- PackagePath

PackagePath es un valor derivado de solo lectura utilizado únicamente para presentar la ubicación del elemento dentro del árbol de paquetes.

También conserva internamente los valores originales de los campos editables. Esto permite determinar si una fila fue realmente modificada mediante `IsDirty`. El formulario trabaja sobre estas copias locales, por lo que editar una celda **no modifica inmediatamente el elemento real de Enterprise Architect**.

El estado `IsDirty` también controla el indicador visual de la grilla. Las celdas editables de una fila modificada se resaltan en amarillo claro. Si los valores vuelven exactamente a su estado original, el indicador desaparece. Después de un guardado exitoso, `AcceptChanges()` establece los nuevos valores como línea base.

Después de un guardado exitoso, `AcceptChanges()` actualiza la línea base local para que el elemento deje de considerarse pendiente.

Persistencia controlada

El repositorio solo se modifica cuando el usuario presiona **Guardar**. En ese momento Addino:

1. finalizar cualquier edición activa;
2. identificar las filas modificadas;
3. si no existen cambios, informar al usuario;
4. **validar todos los Name de las filas dirty mediante `string.IsNullOrEmpty`**;
5. si existe al menos uno inválido, bloquear todo el guardado antes de cualquier `Element.Update()`;
6. recuperar cada elemento mediante `ElementId`;
7. asignar Nombre, Alias y Notas;
8. ejecutar `Element.Update()`;
9. ejecutar `AcceptChanges()` sólo sobre las filas correctamente persistidas;
10. mostrar el resumen de resultado.

El diseño evita ejecutar `Update()` sobre filas que no cambiaron. Como consecuencia, presionar Guardar nuevamente sin introducir nuevas modificaciones devuelve un mensaje indicando que **no existen cambios pendientes**, sin procesar otra vez los elementos ya persistidos.

Manejo de errores

El guardado se realiza de manera independiente por fila. La solución contempla que `Element.Update()` pueda:

- retornar `false`;
- producir una excepción COM;
- fallar por un elemento bloqueado o no modificable.

Un fallo sobre un elemento no debe impedir que Addino continúe intentando procesar las demás filas.

Las filas persistidas correctamente dejan de estar pendientes, mientras que aquellas que presentan errores conservan su estado local para poder ser revisadas o reintentadas.

El comportamiento ante fallos parciales fue validado durante las pruebas funcionales. Cuando una fila no puede persistirse, conserva su estado pendiente, mientras que las filas guardadas correctamente actualizan su línea base mediante `AcceptChanges()`. El procesamiento de las demás filas continúa y el resultado se comunica al usuario.

Como riesgo residual no bloqueante, no se indujo de forma segura un fallo catastrófico durante la reconstrucción completa de la grilla en Recargar. La implementación materializa los nuevos datos antes de reemplazar el contenido visible para preservar el estado anterior ante una excepción.

Archivos principales de la solución

La implementación se distribuye principalmente en:

Addino.sln: Solución de Visual Studio utilizada como punto de entrada para abrir y compilar Addino.

Addino.csproj: Proyecto C# y configuración de compilación.

AddinoClass.cs: Clase COM principal y callbacks utilizados por Enterprise Architect.callbacks COM y loader recursivo compartido.

MetadataElementRow.cs : Modelo local utilizado para representar cada elemento de la grilla y detectar modificaciones. DTO, valores originales, IsDirty y PackagePath.

MetadataReviewForm.cs: Lógica de la ventana, edición, persistencia, cancelación y manejo de errores. edición, validación, Guardar, Recargar, indicadores, cancelación y caption DWM best-effort.

MetadataReviewForm.Designer.cs: Definición visual del formulario Windows Forms. layout, grilla y estilos visuales.

Properties/AssemblyInfo.cs: Configuración del ensamblado y visibilidad COM.

La identidad interna Addino, el namespace, el assembly y la clase COM se mantienen deliberadamente estables para evitar modificar innecesariamente la integración ya validada con Enterprise Architect.

Validación funcional

La solución fue comprobada manualmente utilizando Enterprise Architect 17.1 x64. Las evidencias se encuentran documentadas en:

[Pino_Evidencias_Pruebas_Funcionales_Addino.pdf](#)

La solución fue validada manualmente utilizando Enterprise Architect 17.1 x64. Este documento, reúne **30 pruebas funcionales**, organizadas entre el alcance base y las funcionalidades opcionales implementadas.

- carga correcta de Addino dentro de Enterprise Architect;
- disponibilidad de la acción del menú;
- rechazo de selecciones inválidas;
- apertura con paquetes válidos;
- carga de elementos directos;
- funcionamiento con paquetes vacíos;
- permisos correctos de edición de columnas;
- edición multilínea de Notas;
- cancelación sin persistencia;
- guardado exitoso;
- Guardar sin cambios pendientes;
- reflejo de los valores guardados en Enterprise Architect;
- conservación del último estado efectivamente persistido.
- integración y selección;
- carga y edición básica;
- persistencia/cancelación;
- validación de Nombre;
- recorrido recursivo y **Paquete**;
- lifecycle dirty;
- Recargar;
- UI y regresiones.

Verify relacionó estas pruebas con los requisitos correspondientes y marcó como conformes los comportamientos obligatorios.

Verificación técnica

Además de las pruebas manuales en Enterprise Architect, se realizó una verificación final independiente sobre el proyecto.

La compilación utilizada fue:

```
msbuild Addino.csproj /t:Build /p:Configuration=Debug /p:Platform=x64
```

El resultado fue: **Build exitoso — 0 errores — 0 advertencias**

y se generó: bin\x64\Debug\Addino.dll

El control de integridad mediante git diff --check también finalizó correctamente.

El Verify final concluyó:

- Build: exit 0;
- 0 errores;

- 0 advertencias;
- 6/6 requisitos conformes;
- 32/32 escenarios conformes;
- 33/33 tareas completadas;
- Human Gates HG1–HG5 aprobados;
- Exercise 2 y configuración protegida sin modificaciones.

Después del Verify satisfactorio, el change SDD fue archivado y sus cambios de especificación se consolidaron en el Spec canónico de [ea-metadata-review](#).

El cierre preservó como riesgo residual no bloqueante el escenario HG4-I y mantuvo R-01..R-13 como checklist de regresión formal planificado.

Evidencia y documentación complementaria

La entrega del Ejercicio 1 se complementa con documentos independientes para evitar concentrar toda la información en un único PDF.

[Pino_Guia_Ejecucion_Addino.pdf](#): Guía orientada a una persona que recibe la carpeta del proyecto y necesita preparar, compilar, registrar y utilizar Addino desde cero. La guía deja claro que Addino.sln debe utilizarse junto con el resto del proyecto y no como archivo aislado.

Los siguientes archivos, se encuentran en la ruta: `\Exercise_1_Addin\docs`

[Pino_Evidencias_Pruebas_Funcionales_Addino.pdf](#): Registro de las pruebas realizadas sobre la versión final y funcional, acompañado por capturas reales de Enterprise Architect.

[Pino_Registro_Uso_IA_Ejercicio_1.pdf](#): Registro independiente de las interacciones significativas con herramientas de Inteligencia Artificial, modelos utilizados, estrategias, decisiones y evidencias. También se incluye el archivo `AI_USAGE_LOG.md`, a modo de resumen del PDF.

Video de ejecución

Complementa las capturas del documento de evidencias mostrando el flujo de uso dentro de Enterprise Architect.

Link Video:  [Pino Guia Ejecucion Addino](#)

Conclusión

Durante todo el proceso de desarrollo, trabajar directamente sobre Enterprise Architect, herramienta que he utilizado como usuario, en mis trabajos universitarios, fue muy emocionante. Entender cómo se piensan, crean, e implementan soluciones reales a problemas de la plataforma fue un gran aprendizaje.

No busque solamente que Addino cumpla el objetivo de centralizar la revisión de metadatos de Enterprise Architect, si no que busque que realmente se sintiera como una función integrada.. La solución final integra carga recursiva, trazabilidad de paquete, validación previa al guardado, identificación de modificaciones pendientes, recarga segura y mejoras de interfaz.

La arquitectura mantuvo una separación simple entre integración COM, modelo local e interfaz, preservando la identidad del Add-in y evitando persistencia implícita.

Esta experiencia me motivó a seguir construyendo soluciones que faciliten el día a día de las personas. Entendí que trabajar de forma ordenada, controlada, constantemente revisada, facilita mucho las barreras entre lo que deseo construir, y lo que la Inteligencia Artificial construye.

Muchas gracias por esta oportunidad.

Atte. Ezequiel Pino